CINEC CAMPUS
Beyond A Graduate

ORACLE AS A MULTI-USER DATABASE

BSc (Hons) in Software Engineering

Advanced Database Systems –IT2210

ORACLE AS A MULTI-USER DATABASE

- So far you have mostly used Oracle as if it were a single-user database.
- That is, you only see your tables and you are unaware of any other tables.
- SQL can be used as a Data Control Language (DCL) to set up the access rights on tables.
- Every Oracle user has a username and password.
- Every user owns the tables, views etc. that they create (if they are allowed to create them).
- Each user has a role(s) associated with them.
  - A role is a set of privileges that can be assigned to a user.
  - Database administrators can grant privileges to roles and then assign a role to a particular user.
  - Alternatively, they can assign specific privileges to a user.

CREATE USERS

- Database administrators (DBA) can create users using SQL:
  - CREATE USER ops$0981234 IDENTIFIED BY cupcake;
  - Creates user ops$0981234 and give him/her a password of cupcake
- Passwords may be altered by the ALTER USER command:
  - ALTER USER ops$0981234 IDENTIFIED BY scone;
  - Changes password of user ops$0981234 to scone

USER CREATION

1. Create the user PeterPan12 with the password 1234

```
CREATE USER PeterPan12 IDENTIFIED BY 1234;
```

2. Change the user PeterPan12's password to 5678

```
ALTER USER PeterPan12 IDENTIFIED BY 5678;
```

PROFILES

- Passwords can expire and accounts can be locked after x number of failed logon attempts. The DBA can associate a profile with a user.
- Profiles can enforce:
  - the frequency with which a password must be changed;
  - the number of failed login attempts before an account is locked;
  - the number of days the account will remain locked;
  - the number of days that must pass before you can reuse a password;
  - the number of password changes before a password can be reused.
- The DBA can also set the minimum password length.
  - Oracle recommend that passwords are at least 8 characters long
  - Since Oracle 11g passwords became case-sensitive

ROLES

- A user with just a username and a password cannot do anything, they must be assigned system privileges.
- Oracle provides some built-in roles for different types of users, but now expect users to create their own roles, tailored to the requirements of the system.
  - For example, the DBA could create a role that allows students the ability to create and manage their own tables, but not see anyone else's.
  - On eVision, a role would have been setup that allows you to view your own data, amend certain details, but not create tables

DATABASE PRIVILEGES

- Privileges define the access rights a user has on a database object.
- There are two types of privileges:
  - **System**
    - This allows the user to CREATE, ALTER or DROP database objects.
    - For example: TABLE, VIEW, or TRIGGER
  - **Object**
    - This allows the user to SELECT, INSERT, UPDATE, or DELETE data from the database object to which the privilege applies, such as a table.
    - Other object privileges include: **ALTER, EXECUTE, INDEX or REFERENCES**
      - **ALTER**: alter the object
      - **EXECUTE**: allows user to execute a stored procedure or function
      - **INDEX**: create an index on the table
      - **REFERENCES**: can create or alter a table to define a foreign key

GRANT – SYSTEM PRIVILEGES

- System Privileges are defined using the GRANT command:

```
GRANT privilege, privilege TO user, user [WITH ADMIN OPTION]
```

E.g.

```
GRANT CREATE PROCEDURE TO userX;
```

  - Allows userX to create stored procedures, functions and packages in their own schema.

```
GRANT CREATE ANY PROCEDURE TO superUser;
```

  - Allows superUser to create stored procedures, functions and packages in any schema.

- If the WITH ADMIN OPTION is specified this means that the user may grant the privileges to other users.
- System privileges can be very powerful, so should be granted cautiously.
- See the full list of different types of privileges:

GRANT – OBJECT PRIVILEGES

- Object privileges are defined similarly:

```
GRANT privilege, privilege TO user, user [WITH ADMIN OPTION]
```

E.g.

```
GRANT SELECT ON student TO ops$0981234;
```

- Allows user ops$0981234 to carry out SELECT operations (queries) on table STUDENT.
- If the WITH GRANT OPTION is specified this means that the user may grant the privileges to other users.

```
GRANT SELECT, INSERT ON student TO ops$1012345 WITH GRANT OPTION;
```

- Means user ops$1012345 can also grant SELECT and INSERT privileges to other users.
- They can not, however, grant a privilege they do not have.
- Use with care though – could have unintended side effects, opening up a security hole!

- System and Object privileges can not be defined in the same statement.

REVOKE COMMAND – SYSTEM PRIVILEGE

- Privileges may be taken away using the REVOKE command:

```
REVOKE CREATE TABLE FROM ops$1012345;
```

- System privileges have no cascading effect, e.g.,
  - User dbaJane gives **CREATE TABLE** system privilege to **Jill WITH ADMIN OPTION**;
  - Jill creates a table: jTable and grants **CREATE TABLE** system privilege to **Bob**;
  - **Bob** creates a table: bTable;
  - **dbaJane** revokes **CREATE TABLE** system privilege from Jill;
  - **Jill's** jTable still exists and **Bob's bTable**;
  - **Jill** loses the **CREATE TABLE** privilege, but **Bob** still has it.

EXCERCISE

1. Grant INSERT and UPDATE privileges on the "orders" table to the role "sales_team".

```
GRANT INSERT, UPDATE ON orders TO sales_team;
```

2. Grant EXECUTE privilege on the "calculate_salary" procedure to the user "finance_user".

```
GRANT EXECUTE ON calculate_salary TO finance_user
```

EXCERCISE

1. Grant INSERT and UPDATE privileges on the "orders" table to the role "sales_team".

```
GRANT INSERT, UPDATE ON orders TO sales_team;
```

2. Grant EXECUTE privilege on the "calculate_salary" procedure to the user "finance_user".

```
GRANT EXECUTE ON calculate_salary TO finance_user
```

3. GRANT EXECUTE ON calculate_salary TO finance_user

```
GRANT SELECT ON products TO user1, user2;
```

4. Grant the SELECT privilege on the "employees" table to the user "hr_user" with the option to grant this privilege to other users.

```
GRANT SELECT ON employees TO hr_user WITH GRANT OPTION;
```

QUESTIONS?

THANK YOU